Hands-on 2 – Hibernate XML Configuration Implementation Walkthrough

Objective

The objective of this assignment is to understand the Hibernate Framework, its XML-based configuration, object-relational mapping process, and the core components used to perform database operations. This assignment also explains the role of SessionFactory, Session, Transaction, and commonly used Hibernate methods.

Introduction

Hibernate is a powerful **Object-Relational Mapping (ORM)** framework for Java that simplifies database interaction by mapping Java objects to relational database tables. It eliminates the need for writing extensive JDBC code and automatically generates SQL queries based on Java objects.

Hibernate supports two types of configuration:

1. XML-Based Configuration
2. Annotation-Based Configuration

In XML-based configuration, all database connection details and object-table mappings are defined in XML files. This approach separates configuration from application code, making applications easier to maintain and modify.

Hibernate internally converts Java objects into database records and retrieves database records as Java objects, allowing developers to work entirely with objects instead of SQL statements.

Object-Relational Mapping (ORM) in Hibernate XML Configuration

Object-Relational Mapping (ORM) is the process of mapping Java classes to database tables and Java object attributes to database columns.

In Hibernate XML configuration, ORM is achieved using two XML files:

1. Hibernate Configuration File (hibernate.cfg.xml)

This file is the main configuration file of Hibernate. It contains all the information required to establish a connection with the database and initialize the Hibernate framework.

The configuration file includes:

- Database driver information
- Database URL
- Username and password
- SQL dialect
- Connection pool settings
- Mapping file locations
- Hibernate properties

When the application starts, Hibernate reads this configuration file to establish communication with the database.

2. Hibernate Mapping File (*.hbm.xml)

The mapping file defines how Java classes correspond to database tables.

It specifies:

- Java class name
- Database table name
- Primary key mapping
- Field-to-column mapping
- Relationship mapping between tables

Hibernate reads this file and automatically performs object-relational mapping during runtime.

Working of Hibernate XML Configuration

The working process of Hibernate using XML configuration is as follows:

1. The application starts and loads the Hibernate configuration file.
2. Hibernate establishes a connection with the relational database.
3. A SessionFactory object is created.
4. SessionFactory creates a Session object.
5. A transaction is initiated.
6. Database operations such as insert, update, retrieve, or delete are performed using the Session object.
7. If all operations are successful, the transaction is committed.
8. If any exception occurs, the transaction is rolled back.
9. Finally, the Session is closed.

This process ensures efficient and reliable interaction with the database.

Core Components of Hibernate

1. SessionFactory

Definition

SessionFactory is the factory class responsible for creating Session objects.

Features

- Created only once during application startup.
- Thread-safe.
- Stores Hibernate configuration and metadata.
- Responsible for creating Session objects.
- Shared throughout the application.

Purpose

The primary purpose of SessionFactory is to establish communication between the application and the database by providing Session instances whenever required.

Advantages

- Improves application performance.
- Avoids repeated loading of configuration files.
- Reduces object creation overhead.
- Maintains metadata of mapped entities.

2. Session

Definition

A Session represents a single connection between the application and the database.

Features

- Lightweight object.
- Created by SessionFactory.
- Not thread-safe.
- Used for executing database operations.

Responsibilities

A Session object performs all persistence-related operations such as:

- Saving objects
- Updating objects
- Retrieving objects
- Deleting objects
- Executing HQL queries

- Managing transactions

Each Session is generally created for a single unit of work.

3. Transaction

Definition

A Transaction represents a sequence of one or more database operations that are treated as a single logical unit.

Purpose

Transactions ensure:

- Data consistency
- Data integrity
- Reliable execution of database operations

If all operations execute successfully, the transaction is committed. Otherwise, all changes are rolled back.

Hibernate Methods

1. beginTransaction()

Purpose

The beginTransaction() method starts a new transaction before performing any database operation.

Importance

- Marks the beginning of a transaction.
- Groups multiple database operations together.
- Ensures atomic execution.

Without beginning a transaction, database modifications cannot be safely committed.

2. commit()

Purpose

The commit() method permanently saves all changes made during the transaction into the database.

Importance

- Makes all database modifications permanent.
- Completes the transaction successfully.
- Ensures data persistence.

Commit is executed only if all operations complete without errors.

3. rollback()

Purpose

The rollback() method cancels all operations performed during the current transaction.

Importance

- Restores the database to its previous consistent state.
- Prevents incomplete or incorrect data updates.
- Maintains database integrity.

Rollback is generally executed whenever an exception occurs before committing the transaction.

4. session.save()

Purpose

The save() method is used to insert a new object into the database.

Working

When invoked:

- Hibernate maps the object to its corresponding table.
- Generates an SQL INSERT statement.
- Stores the object permanently in the database after the transaction is committed.

Benefits

- Simplifies insertion operations.
- Eliminates manual SQL coding.
- Automatically manages object persistence.

5. session.createQuery().list()

Purpose

This method is used to retrieve multiple records from the database using Hibernate Query Language (HQL).

Working

- Executes an HQL query.
- Converts the HQL query into SQL.
- Retrieves matching records.
- Returns the result as a Java List.

Benefits

- Simplifies data retrieval.
- Supports object-oriented querying.
- Eliminates manual ResultSet processing.

6. session.get()

Purpose

The get() method retrieves a single object using its primary key.

Working

- Searches the database for the specified primary key.
- Returns the corresponding object if found.
- Returns null if no matching record exists.

Benefits

- Efficient retrieval using primary keys.
- Simple and reliable data access.

7. session.delete()

Purpose

The delete() method removes an existing object from the database.

Working

- Identifies the object's primary key.
- Generates the SQL DELETE statement.
- Removes the corresponding record from the database.

Benefits

- Simplifies deletion operations.
- Automatically handles SQL generation.
- Maintains database consistency.

Advantages of Hibernate XML Configuration

- Provides clear separation between configuration and application code.
- Simplifies object-relational mapping.
- Eliminates repetitive JDBC programming.
- Automatically generates SQL statements.
- Supports transaction management.
- Improves code readability and maintainability.
- Enables database independence.
- Supports multiple relational database systems.
- Simplifies CRUD operations.
- Reduces application development time.